

Final report for high dimensional probability class by Ben Lechner id 214764946

Chapter 1: Objective

This project's goal is to detect a song's genre using a group of predetermined features. Music genres don't have clear criteria and that makes them relatively hard to categorize, especially because fusions between certain genres exist. While genres don't have clear definitions, there are features that we can rely on to determine a song's genre. Pop music tends to be upbeat with a simple chord progression, rock music is associated with playing distorted power chords on the electric guitar and playing guitar solos in the bridge, jazz music leans on improvisations and harmonic complexity and so on. My objective is to correctly detect a song's genre using features like the ones I mentioned above.

Chapter 2: Data Collection

Link to the dataset I used in this project:

<https://www.kaggle.com/code/jessieangelica/spotify-tracks-dataset>

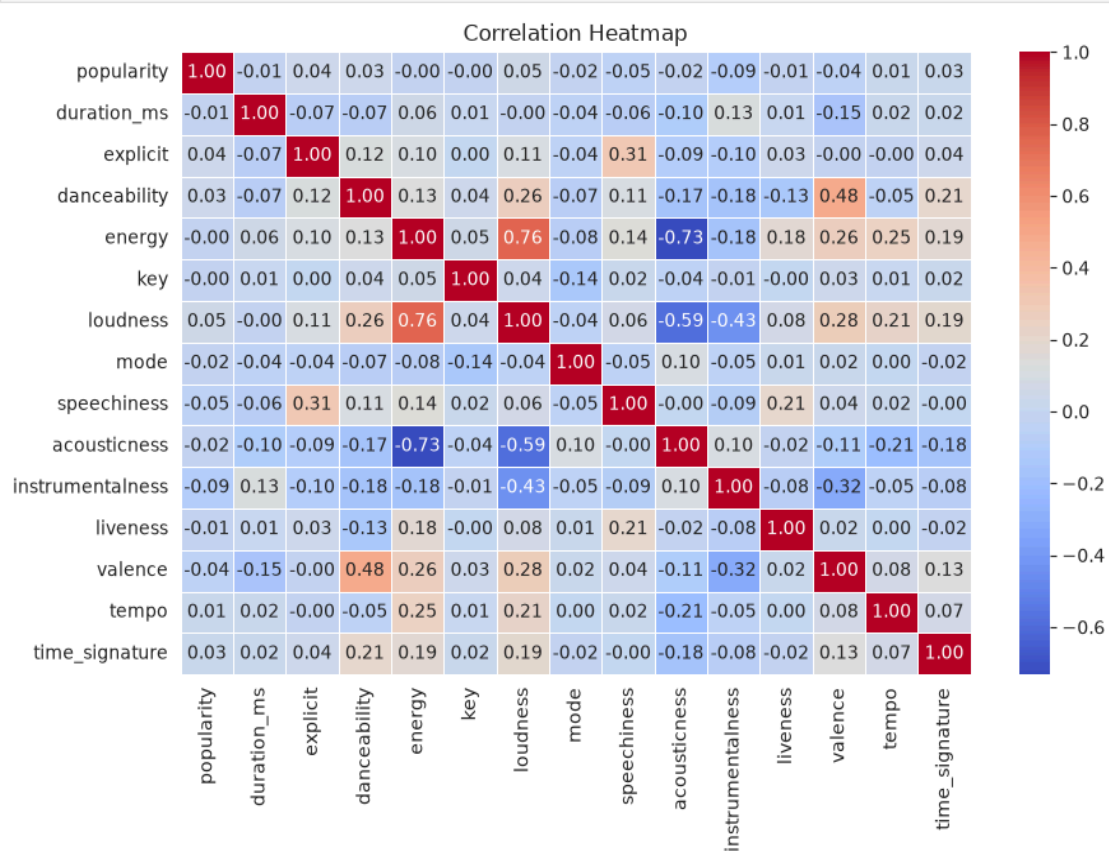
The dataset I chose is called spotify-tracks-dataset which was sourced from Kaggle. It consists of 114,000 songs collected from the official spotify web API. The dataset pairs track data such as song title, artist, and album with audio features extracted through Spotify's algorithms. Each track is labeled with its genre which means I'm dealing with a supervised learning task. The data is split into training and testing sets - 80% train, 20% test.

The dataset features are as follows:

- **track_id**, **track_name**, **artists**, **album_name**
- **track_genre**
- **explicit**: A boolean flag (True/False) indicating whether the track contains explicit lyrics.
- **duration_ms**: The total duration of the track measured in milliseconds.
- **popularity**: An integer score ranging from 0 to 100 that measures the stream count and recent listening trends of the track relative to all other tracks on the platform.
- **danceability** (0.0 - 1.0): Describes how suitable a track is for dancing based on a combination of musical elements including tempo, rhythm stability, beat strength, and overall regularity.
- **energy** (0.0 - 1.0): A perceptual measure of intensity and activity. Highly energetic tracks (e.g., Heavy Metal) feel fast, loud, and noisy, whereas acoustic classical pieces yield lower scores.
- **valence** (0.0 - 1.0): A measure describing the musical positiveness conveyed by a track. High valence tracks sound

more happy or euphoric, while low valence tracks sound sad or aggressive.

- **acousticness** (0.0 - 1.0): A confidence measure from 0.0 to 1.0 of whether the track is acoustic.
- **instrumentalness** (0.0 - 1.0): Predicts whether a track contains no vocals. Values above 0.5 indicate instrumental tracks, with values closer to 1.0 representing pure instrumental compositions.
- **speechiness** (0.0 - 1.0): Detects the presence of spoken words in a track. Values between 0.33 and 0.66 represent speech and music combinations (e.g., Rap music), while values below 0.33 indicate traditional music tracks.
- **liveness** (0.0 - 1.0): Detects the presence of an audience in the recording. Higher values represent a higher probability that the track was recorded live.
- **loudness**: The overall loudness of a track in decibels (dB), averaged across the entire track.
- **tempo**: The overall estimated tempo of a track in beats per minute (BPM).
- **key**: The key the track is in, mapped to standard Pitch Class notation 0 =C, 1 = C#/Db, 11 = B.
- **mode**: Indicates the modality (Major = 1, Minor = 0) of the track.
- **time_signature**: An estimated overall time signature specifying how many beats are in each bar.



Chapter 3: Data Filtering

To enhance dataset quality and prevent extreme outliers from distorting model training, I applied a duration based filtering based on the track length (`duration_ms`). Tracks with durations shorter than 30 seconds or longer than 10 minutes were identified as non standard instances such as brief audio interludes, sound effects, or uncharacteristically extended compositions that do not represent typical musical structures. Applying these boundary conditions led to the removal of 620 anomalous tracks, refining the dataset from its initial 114,000 samples to a cleaned subset of 113,380 tracks while preserving the overall statistical distribution of the feature space. An alternative idea of filtering the extreme 1% of the data was considered but a manual cut based on domain knowledge is better than a statistical cut.

```
print('Updated dataset size: %d tracks.' % len(dataset))
```

```
Removed 620 anomalous tracks (shorter than 30 seconds or longer than 10 minutes).  
Updated dataset size: 113380 tracks.
```

Chapter 4: The Machine Learning Models I used

In this project 3 models were used, each one is more complicated than the other in order to increase the accuracy, precision, recall and f1 score of the project

My baseline model: KNN

KNN serves as the foundational baseline model. As a non parametric, distance based algorithm KNN relies directly on spatial relationships, making it the most theoretical reference point for evaluating how distance preserving random projections affect decision boundaries in lower dimensions.

Random Forest:

Introduces higher model complexity via an ensemble of unpruned decision trees using bootstrap aggregation (bagging). This allows for capturing non linear feature interactions and evaluating how axis aligned decision splits behave when high dimensional audio features are projected into lower dimensional subspaces.

XGBoost:

Represents the state of the art in gradient boosted decision trees, pushing model capacity to its peak for tabular datasets. By iteratively optimizing objective loss functions and utilizing regularized boosting, XGBoost evaluates whether advanced ensemble methods can overcome potential information loss introduced by random projections and maximize total classification accuracy.

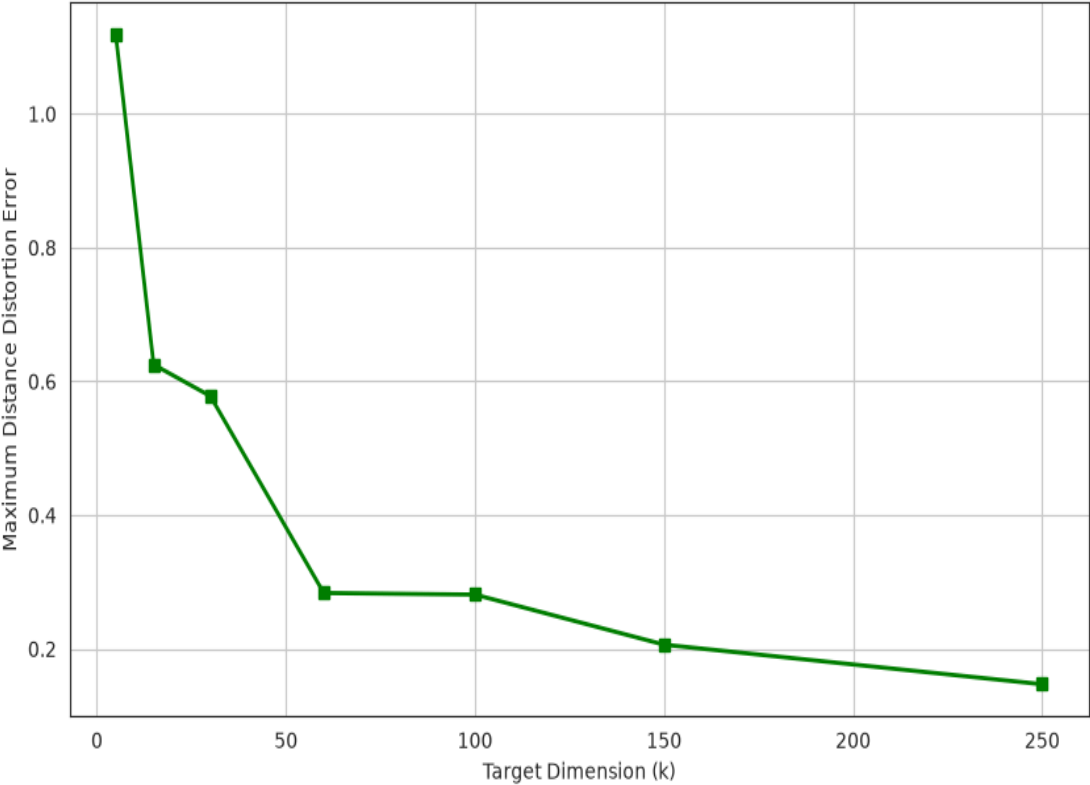
Chapter 5: JL dimensionality reduction

To evaluate the operational impact of high dimensional compression as required by the project specifications, the dataset underwent linear dimensionality reduction via the JL transform.

The experimental pipeline was designed around a controlled comparative strategy:

1. Baseline Evaluation (Original Feature Space): All three classification models (KNN, Random Forest, and XGBoost) were initially trained and evaluated on the full, unreduced feature matrix (d dimensions). This established the upper bound benchmark for classification metrics and computational runtime.
2. Random Projection (Reduced Subspace): Using a random projection matrix $R \in \mathbb{R}^{d \times k}$, the original feature space was mapped into a lower dimensional subspace ($k < d$), preserving pairwise euclidean distances within an epsilon distortion factor.
3. Comparative Performance & Efficiency Trade offs: The compressed dataset was subsequently used to train identical instances of the three algorithms. The primary focus of this setup is to quantitatively analyze the resulting trade offs across two main axes: predictive accuracy (assessing how distance preservation translates to class boundary retention across distance based vs tree based models) and computational efficiency (measuring speedups in training and inference time alongside memory optimization).

Johnson-Lindenstrauss Lemma: Distance Preservation vs. Dimension k



Chapter 6: Execution and Methodology

The dataset was first filtered to include five target genres: Rock, Pop, Jazz, Country, and Acoustic. Each song was represented by a set of audio characteristics extracted by Spotify, including danceability, energy, loudness, speechiness, acousticness, instrumentalness, liveness, valence, tempo, and popularity. These features served as the input variables, while the corresponding song genre was used as the target label. The dataset was divided into training and testing sets using an 80/20 split while preserving the class distribution through stratified sampling. Three supervised machine learning algorithms were evaluated: KNN, Random Forest, and XGBoost. Each model was first trained and evaluated on the original feature space. Next the JL random projection was applied to reduce the dimensionality of the feature vectors while approximately preserving pairwise distances. The same three models were then retrained and evaluated on the transformed dataset. Model performance before and after the JL transformation was compared using accuracy, precision, recall, and f1 score allowing us to assess both the classification capabilities of the different algorithms and the impact of dimensionality reduction on their performance.

Chapter 7: Results

The table below summarizes the quantitative evaluation metrics (accuracy, precision, recall, and macro f1 Score) for KNN, Random Forest, and XGBoost across both the unreduced feature space and the JL projected subspace

	Model	Accuracy	Precision	Recall	F1
0	KNN (Original)	0.649	0.650	0.649	0.645
1	RF (Original)	0.775	0.779	0.775	0.772
2	XGBoost (Original)	0.782	0.781	0.782	0.781
3	KNN (JL)	0.645	0.647	0.645	0.640
4	RF (JL)	0.710	0.712	0.710	0.708
5	XGBoost (JL)	0.708	0.707	0.708	0.705

[]:

Impact of the JL Transform on each model:

KNN

Accuracy dropped marginally by only 0.4% (from 64.9% to 64.5%), with f1 score remaining virtually stable (0.645 to 0.640)

A possible explanation for that may be because KNN relies strictly on pairwise Euclidean distances (L2 norm) between data points to assign class membership, it aligns directly with the mathematical guarantees of the JL Lemma. The random projection matrix preserves relative spatial distances between feature vectors within an ϵ bound, enabling KNN to maintain its exact decision boundaries even in the compressed space.

Random Forest and XGBoost

Both tree based models experienced a noticeable performance drop of 6.5% and 7.4%. Random Forest accuracy decreased from 77.5% to 71.0% and XGBoost accuracy decreased from 78.2% to 70.8%.

Decision trees construct decision boundaries by executing axis aligned orthogonal splits on individual features. In the original feature space, tree models heavily leveraged primary indicators like popularity (importance score of 0.230 in RF and 0.322 in XGBoost) and acousticness (0.126 in RF and 0.148 in XGBoost). The JL linear transformation mixes all original features into random linear combinations. This obfuscates single feature importance, forcing tree classifiers to attempt orthogonal splits on arbitrary, rotated linear projections thereby lowering classification precision across complex genres like country and rock.

Overall XGBoost achieved the highest absolute classification performance on the original dataset (78.2% accuracy), followed closely by Random Forest (77.5%). While dimensionality reduction via random projection incurs a modest accuracy penalty for tree ensembles, it offers significant memory efficiency and computational speedups, while preserving spatial distance structures almost perfectly for instance based learners like KNN.

```

===== KNN (Original Data) =====
precision    recall  f1-score   support

 acoustic    0.69    0.81    0.75     200
 country     0.56    0.55    0.55     200
  jazz       0.79    0.67    0.72     200
   pop       0.62    0.74    0.67     199
  rock       0.60    0.47    0.52     200

 accuracy          0.65     999
 macro avg         0.65    0.65    0.64     999
weighted avg         0.65    0.65    0.64     999

```

```

[[163  9 12 11  5]
 [ 28 110 12 23 27]
 [ 27 24 134  6  9]
 [ 11 16  3 148 21]
 [  8 38  9 52 93]]

```

```

===== Random Forest (Original Data) =====
precision    recall  f1-score   support

 acoustic    0.73    0.92    0.81     200
 country     0.75    0.61    0.67     200
  jazz       0.88    0.79    0.83     200
   pop       0.76    0.83    0.80     199
  rock       0.78    0.73    0.75     200

 accuracy          0.77     999
 macro avg         0.78    0.77    0.77     999
weighted avg         0.78    0.77    0.77     999

```

```

[[183 10  2  4  1]
 [ 26 123 12 17 22]
 [ 28  7 157  7  1]
 [  7  8  2 165 17]
 [  8 17  6 23 146]]

```

```

--- Feature Importances (Random Forest) ---
Importance of danceability: 0.084
Importance of energy: 0.100
Importance of loudness: 0.096
Importance of speechiness: 0.088
Importance of acousticness: 0.126
Importance of instrumentalness: 0.053
Importance of liveness: 0.073
Importance of valence: 0.078
Importance of tempo: 0.071
Importance of popularity: 0.230

```

```

===== KNN (JL Projection) =====
      precision    recall  f1-score   support

 acoustic      0.68      0.80      0.73      200
 country      0.55      0.54      0.54      200
 jazz         0.77      0.69      0.72      200
 pop          0.60      0.74      0.67      199
 rock         0.63      0.47      0.53      200

 accuracy              0.64      999
 macro avg           0.65      0.64      0.64      999
 weighted avg       0.65      0.64      0.64      999

[[159  11  12  13   5]
 [ 27 107  14  26  26]
 [ 28  21 137   6   8]
 [ 10  20   5 148  16]
 [  9  36  10  52  93]]

===== Random Forest (JL Projection) =====
      precision    recall  f1-score   support

 acoustic      0.74      0.84      0.79      200
 country      0.67      0.58      0.62      200
 jazz         0.86      0.78      0.82      200
 pop          0.64      0.75      0.69      199
 rock         0.65      0.59      0.62      200

 accuracy              0.71      999
 macro avg           0.71      0.71      0.71      999
 weighted avg       0.71      0.71      0.71      999

[[169   8  10   9   4]
 [ 24 116   7  25  28]
 [ 21   9 155   9   6]
 [  7  14   2 150  26]
 [  8  27   6  40 119]]

```

```

===== XGBoost (Original Data) =====
              precision    recall  f1-score   support

    acoustic      0.78      0.85      0.82      200
    country      0.74      0.70      0.72      200
    jazz          0.85      0.84      0.85      200
    pop           0.77      0.80      0.79      199
    rock          0.76      0.71      0.73      200

    accuracy              0.78      999
    macro avg      0.78      0.78      0.78      999
    weighted avg   0.78      0.78      0.78      999

```

```

[[170  13  10   6   1]
 [ 12 141  12  14  21]
 [ 16   6 169   5   4]
 [ 11   9   1 159  19]
 [  8  22   6  22 142]]

```

```

--- Feature Importances (XGBoost) ---
Importance of danceability: 0.067
Importance of energy: 0.072
Importance of loudness: 0.072
Importance of speechiness: 0.075
Importance of acousticness: 0.148
Importance of instrumentalness: 0.074
Importance of liveness: 0.058
Importance of valence: 0.060
Importance of tempo: 0.051
Importance of popularity: 0.322

```

```

print(confusion_matrix(y_true, y_pred_ensemble))

```

```

===== XGBoost (JL Projection) =====
              precision    recall  f1-score   support

    acoustic      0.75      0.84      0.80      200
    country      0.66      0.56      0.61      200
    jazz          0.82      0.79      0.80      200
    pop           0.64      0.76      0.70      199
    rock          0.67      0.58      0.62      200

    accuracy              0.71      999
    macro avg      0.71      0.71      0.70      999
    weighted avg   0.71      0.71      0.70      999

```

```

[[169   9   9   9   4]
 [ 21 113  13  23  30]
 [ 20   8 157  12   3]
 [  6  12   8 152  21]
 [  8  30   5  41 116]]

```

```

[351]: def get_metrics(name, y_true, y_pred):

```

Chapter 8: Conclusions

In conclusion, this project demonstrated the trade offs between model complexity and dimensionality reduction using the JL transform on Spotify audio data. While gradient boosted trees (XGBoost) achieved the highest overall accuracy (78.2%), tree based ensembles experienced a moderate performance decline under random projection due to the distortion of axis aligned feature splits. Conversely, distance based classifiers like KNN proved exceptionally resilient to JL reduction, maintaining near identical accuracy (64.5%). Ultimately, the choice to implement random projection depends on system priorities. It provides a powerful mechanism for drastically reducing computational overhead and memory usage with minimal impact on distance dependent algorithms, though at a slight cost to non linear tree models.